젤리비 AGV2 대회 노트북 세팅 · 사용 가이드

새 노트북(Windows)에서 처음부터 대회 당일까지 필요한 순서를 모두 적었다. **1~7단계는 대회 전날 끝내 둔다.** 8단계부터가 대회장에서 하는 일이다.

---

0. 준비물

- ☐ 대회용 노트북 + 충전기
- ☐ 로봇용 USB 케이블 (지금까지 업로드에 쓰던 것. 여분 1개 있으면 좋음)
- ☐ 로봇 배터리 완충 + 여분
- ☐ 노트북 인터넷 (현장 와이파이가 불안하면 휴대폰 핫스팟)
- ☐ GitHub 계정 ID/비밀번호 (저장소 주인 계정)
- ☐ Claude 계정 (claude.ai 로그인)

---

1. Arduino IDE 설치

1. https://www.arduino.cc/en/software 에 들어간다.
2. **Arduino IDE 2.x → Windows (Win 10 and newer, 64 bits)** 를 받는다. 기부 화면이 나오면 **JUST DOWNLOAD**를 누른다.
3. 설치 중 드라이버 설치를 물으면 모두 **설치**를 누른다.
4. 실행해서 첫 화면이 뜨면 완료.

2. CH341 USB 드라이버 설치 (로봇 인식용)

젤리비 보드는 CH340/CH341 USB 칩을 쓴다. 드라이버가 없으면 포트가 안 보인다.

1. https://www.wch-ic.com/downloads/CH341SER_EXE.html 에서 **CH341SER.EXE**를 받는다.
2. 실행 → **INSTALL** → "Driver install success" 가 나오면 완료.
3. 로봇을 USB로 연결하고 **로봇 전원 ON**.
4. 시작 버튼 우클릭 → **장치 관리자** → **포트(COM & LPT)** 에 `USB-SERIAL CH340 (COM3)` 처럼 보이면 성공. **COM 번호를 기억해 둔다.**

3. GitHub Desktop 설치 + 로그인

코드는 GitHub에 있다. Claude가 고친 코드를 노트북으로 받아오는 프로그램이다.

1. https://desktop.github.com/download/ 에서 설치한다.
2. 실행 → **Sign in to GitHub.com** → 브라우저에서 저장소 주인 계정으로 로그인 → **Authorize**.
3. 이름·이메일 설정 화면은 그대로 **Finish**.

4. 코드 받기 (clone)

1. GitHub Desktop → **File → Clone repository** → **GitHub.com** 탭.
2. 목록에서 **`p6w94kdjhy-bit/-`** 를 고른다. (안 보이면 **URL** 탭에 `https://github.com/p6w94kdjhy-bit/-` 입력)
3. **Local path** 를 `C:\robot\-` 로 하고 **Clone**.
4. 위쪽 **Current branch** 를 눌러 **`claude/jellibi-agv2-contest-rnqx70`** 를 선택한다. ← 꼭 이 브랜치
5. 탐색기에서 `C:\robot\-\jellibi_agv2` 폴더가 있고 그 안에 `course1_drive`, `course2_pallet`, `calib_ir`, `tune_drive`, `libraries` 가 있으면 성공.

웹에서 코드 보기: https://github.com/p6w94kdjhy-bit/-/tree/claude/jellibi-agv2-contest-rnqx70/jellibi_agv2

5. Arduino IDE 설정

5-1. 스케치북 위치 (가장 중요)

1. Arduino IDE → **File(파일) → Preferences(기본 설정)**.
2. **Sketchbook location(스케치북 위치)** 을 `C:\robot\-\jellibi_agv2` 로 바꾸고 OK.
3. **IDE를 완전히 껐다 켠다.** → 이렇게 해야 공용 코드(`libraries/AGV2` , 보정값 `config.h`)를 IDE가 찾는다.

5-2. 보드 패키지 · Servo 라이브러리

1. 왼쪽 **보드 매니저**(두 번째 아이콘) → `Arduino AVR Boards` 검색 → 설치되어 있지 않으면 **INSTALL**.
2. 왼쪽 **라이브러리 매니저**(세 번째 아이콘) → `Servo` 검색 → **Servo by Arduino** 가 INSTALLED 가 아니면 **INSTALL**.

5-3. 보드 · 포트 선택

- **Tools(툴) → Board → Arduino AVR Boards → Arduino Nano**
- **Tools → Processor → ATmega328P** (업로드가 실패하면 **ATmega328P (Old Bootloader)** 로 바꾼다. 맞는 쪽을 기억해 둘 것)
- **Tools → Port → COM3** (2단계에서 본 번호)

6. 첫 컴파일 · 업로드 테스트

1. **File → Sketchbook → calib_ir** 를 연다. (또는 File → Open → `C:\robot\-\jellibi_agv2\calib_ir\calib_ir.ino`)
2. 왼쪽 위 **✓ (Verify)** → 아래에 `Sketch uses ... bytes` 가 나오면 성공.
3. 로봇 전원 ON 상태에서 **→ (Upload)** → `Done uploading` 이면 성공.
4. 오른쪽 위 **돋보기(Serial Monitor)** → 아래쪽 속도를 **115200 baud** 로. `L=... R=... F=...` 숫자가 계속 나오면 완료. 손으로 센서를 가리면 숫자가 바뀐다.

나머지 3개 스케치(`course1_drive` , `course2_pallet` , `tune_drive`)도 **✓ Verify** 만 눌러 오류가 없는지 확인해 둔다.

7. Claude 작업 창 열어두기

- 이 대화(세션): https://claude.ai/code/session_016MkBxvgMtn2TAvaizhZMSu
- 세션 목록: https://claude.ai/code
- 새 노트북 브라우저에서 위 주소로 들어가 같은 대화를 이어간다. 대회 당일에는 탭을 열어 둔다.

---

8. 폴더와 파일 — 무엇을 어디서 고치나

```
C:\robot\-\jellibi_agv2\
├─ libraries\AGV2\src\config.h   ★ 보정값 전부 (명도, 전방값, 모터비율, 속도, 회전시간, 리프트)
├─ calib_ir\          현장 IR 명도 측정
├─ tune_drive\        직진·회전·정차 위치 보정 테스트
├─ course1_drive\     종목① 지정경로 순회정차
├─ course2_pallet\    종목② 팔레트 운반 (13:20 제출본)
├─ README.md          전체 설명
└─ CHANGELOG.md       보정값 바꾼 기록
```

직접 수정할 파일은 `config.h` 하나뿐이다. Arduino IDE에서 스케치를 열면 `config.h` 는 탭에 없다. 탐색기에서 `C:\robot\-\jellibi_agv2\libraries\AGV2\src\config.h` 를 **메모장**(또는 IDE의 File → Open)으로 열어 숫자를 고치고 저장한다. **고친 뒤에는 반드시 주행 스케치를 다시 업로드**해야 로봇에 반영된다.

9. 현장 순서

9-1. 명도 측정 (calib_ir)

1. calib_ir 업로드 → 시리얼 모니터 115200.
2. 대회장 조명 아래, 맵 위에서 로봇을 놓고 시리얼 모니터 입력칸에 글자를 입력 → Enter:

입력	로봇 놓는 법
`w`	두 바닥 센서가 **흰 바닥** 위
`b`	로봇을 선과 **직각**으로 놓아 두 센서가 모두 **선 정중앙** 위 (파란 선을 따라 달리면 파란 선에서)
`c`	로봇을 **선 방향**으로, 선 한가운데 (주행하는 자세)
`f`	장애물을 **멈추고 싶은 거리**에 두고
`p`	config.h에 넣을 줄 출력

1. `p` 결과를 **Claude에게 그대로 복사해 보내거나**, 직접 `config.h` 맨 위 [1][2] 의 같은 이름 줄을 바꾼다.
2. 기준값 (흰+검)/2 는 자동 계산된다.
3. 값이 `-1` 이면 주행 스케치는 출발하지 않고 낮은 경고음만 낸다.
4. 경고 줄(`XING_LEVEL_PCT` , `LT_CENTER_OFFSET`)이 나오면 그 값도 반영.

9-2. 주행 보정 (tune_drive)

업로드 후 시리얼 모니터에 글자 입력. 한 번 입력한 명령은 **로봇 버튼으로 반복** 실행된다 (케이블 빼고 테스트 가능).

순서	입력	보는 것	안 맞으면 바꿀 값
1	`s`	앞으로 가나 / 똑바로 가나	뒤로 가면 `LEFT/RIGHT_FWD_LEVEL`, 휘면 `MOTOR_L/R_GAIN`
2	`L`	제자리 정확히 한 바퀴	`TURN90_MS`
3	`l` `r` `u`	좌90·우90·U턴 후 선 위에 정확히	`TURN_BLIND_PCT`, `TURN_EXIT_MAX_MS`
3-1	`F`	선을 따라 똑바로 가나 (지그재그·이탈)	`LT_INNER_PCT`
4	`f`	교차점 1칸 후 정지 — 바퀴축 중점이 교차점 ±30mm	`ALIGN_MS` (앞에 서면 줄이고 못 미치면 늘림)
5	`F`	3칸 갈 때 부저 틱이 정확히 3번	더 울리면 `XING_SKIP_MS` ↑ 또는 `XING_LEVEL_PCT` ↑
6	`q`	사각형 돌고 제자리	종합
7	`p` `d`	리프트 올림/내림 높이	`LIFT_UP_ANGLE`, `LIFT_DOWN_ANGLE`
8	`x`	팔레트 밑에서 후진으로 빠져나옴	`EXIT_BACK_MS`

`f` · `F` 는 로봇을 교차점 **바로 앞**에서 선 방향으로 놓고 시작, `q` 는 바퀴축이 **교차점 위**에 오도록 놓고 시작한다 (오른쪽으로 사각형).

9-3. 경기 실행

1. `course1_drive` 또는 `course2_pallet` 업로드.
2. 출발 위치에 놓으면 **도-미-솔** (준비 완료).
3. 신호 후 **버튼을 눌렀다 떼면** 바로 출발 (5초 안에 출발선 통과).
4. 주행 중 **버튼 = 비상정지**.
5. 끝나면 높은 음 2번.

소리	의미
도-미-솔	준비 완료, 버튼 기다림
짧은 틱	교차점 1개 셈
정차 시 삑	2초 정차 시작
낮은 음 1초마다 반복	오류 (명도값 미입력 등) — 시리얼 모니터에 이유 표시
중간 음 1초마다 반복	비상정지됨

9-4. 속도 올리기

모든 동작이 안정된 뒤 `SPEED_CRUISE` 부터 10씩 올린다. `SPEED_APPROACH` 를 바꾸면 `ALIGN_MS` 를 다시 맞춘다.

10. Claude와 주고받는 방법

```
나: 상황·측정값·테스트 결과 전달
 → Claude: 코드/config.h 수정 + 컴파일 확인(+경로 시뮬레이션) + GitHub에 올림
 → 나: GitHub Desktop에서 받기 → Arduino IDE로 업로드 → 테스트
 → 결과 전달 → 반복
```

받는 법 (Claude가 "올렸다"고 하면)

1. GitHub Desktop → **Fetch origin** → 버튼이 **Pull origin** 으로 바뀌면 누른다.
2. Arduino IDE에서 스케치 탭을 **닫았다가 다시 연다** (예전 내용이 보일 수 있음).
3. 업로드.

보내는 법 — 이렇게 쓰면 한 번에 고친다

- **어디서**: "F5에서 좌회전할 때", "C5 정차할 때"
- **어떻게**: "90도보다 덜 돌고 멈춤", "교차점을 두 번 셈(틱 2번)", "정차가 교차점보다 3cm 앞"
- **측정값**: calib_ir `p` 출력 그대로 복사
- **로그**: USB 연결 상태로 돌렸다면 시리얼 모니터 내용 복사 (지나간 교차점이 찍혀 있음)
- **사진**: 맵, 팔레트, 장애물 위치

노트북에서 config.h를 직접 고친 경우

- 고친 값을 Claude에게도 알려준다 (Claude 쪽에도 반영해야 다음 받기 때 충돌이 없음).
- Pull 할 때 충돌이 나면: GitHub Desktop **Changes** 탭에서 `config.h` 우클릭 → **Discard changes** → 다시 **Pull origin**.

---

11. 13:20 운반 프로그램 제출

- 제출 전 마지막 버전을 고정: GitHub Desktop → **History** 탭 → 맨 위 커밋 우클릭 → **Create Tag** → `submit-course2` → 위쪽 **Push origin**. (또는 Claude에게 "제출본 태그 달아줘")
- **제출 후에는 `course2_pallet` 코드· `libraries` 코드는 수정 금지.** `config.h` 의 승인된 보정값만 바꾼다 → `CHANGELOG.md` 에 기록.
- 종목① (`course1_drive`) 은 계속 개선 가능. 단 공용 라이브러리를 고치면 종목②에도 영향을 주므로 제출 후엔 라이브러리를 건드리지 않는다.

---

12. 문제 해결

증상	해결
포트(COM)가 안 보임	로봇 전원 ON, 다른 USB 케이블(충전 전용 케이블 X), CH341 드라이버 재설치
업로드 실패 `not in sync` / `programmer is not responding`	Processor 를 **ATmega328P (Old Bootloader)** 로 변경, 포트 확인, 시리얼 모니터 닫고 재시도
`AGV2.h: No such file or directory`	5-1 스케치북 위치가 `jellibi_agv2` 폴더인지 확인 → **IDE 재시작**
`Servo.h: No such file`	5-2 라이브러리 매니저에서 Servo 설치
시리얼 모니터 글자 깨짐	속도를 **115200** 으로
출발 안 하고 낮은 경고음 반복	`config.h` 명도값이 `-1` (미입력). 종목②는 `FRONT_OBSTACLE_TH` 도 필요
버튼 눌러도 반응 없음	버튼을 **눌렀다 떼야** 출발
로봇이 뒤로 감 / 제자리 돎	`LEFT_FWD_LEVEL` , `RIGHT_FWD_LEVEL` (HIGH↔LOW)
받기(Pull) 했는데 IDE에 옛날 코드	스케치 탭 닫고 다시 열기
GitHub Desktop에 브랜치가 다름	Current branch → `claude/jellibi-agv2-contest-rnqx70`

13. 전날 최종 체크리스트

- ☐ Arduino IDE 설치, 실행됨
- ☐ CH341 드라이버 설치, 장치 관리자에 COM 포트 보임
- ☐ GitHub Desktop 로그인, 저장소 clone, 브랜치 `claude/jellibi-agv2-contest-rnqx70`
- ☐ 스케치북 위치 = `C:\robot\-\jellibi_agv2` , IDE 재시작
- ☐ Arduino Nano / ATmega328P (또는 Old Bootloader) / COM 포트 선택
- ☐ 스케치 4개 모두 ✓ Verify 성공
- ☐ calib_ir 업로드 → 시리얼 모니터에 숫자 나옴
- ☐ Fetch → Pull 한 번 연습
- ☐ config.h 파일 위치 확인 (메모장으로 열어 보기)
- ☐ Claude 세션 링크 열림
- ☐ 배터리 충전

링크 모음

용도	주소
Arduino IDE	https://www.arduino.cc/en/software
CH341 드라이버	https://www.wch-ic.com/downloads/CH341SER_EXE.html
GitHub Desktop	https://desktop.github.com/download/
(선택) Git 명령어	https://git-scm.com/download/win
코드 (웹)	https://github.com/p6w94kdjhy-bit/-/tree/claude/jellibi-agv2-contest-rnqx70/jellibi_agv2
Claude 세션	https://claude.ai/code/session_016MkBxvgMtn2TAvaizhZMSu
참고 예제 (RobotNMore)	https://github.com/RobotNMore/JellibiAGV